Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 9: **Data Processing: Import, Export, and Queues**

From summary reports and CSV files to heavy work handed off to the background

What You'll Learn

1. Build a report that summarizes data over a date range, and avoid a common pitfall when filtering by date
2. Export data to CSV without overloading server memory, and import CSV while handling bad rows without stopping the whole process
3. Recognize when a job should be handed off to a **queue** instead of being done while the user waits

> These slides cover the concepts. Applying reports, export/import, and queues to Simple POS is done in the lab jobsheet.

Meeting Map: Think of a Restaurant Kitchen

Raw data → **Report** → **CSV out / in** → **Queue**

> Picture a restaurant kitchen. Raw ingredients come in (**extract**), get cooked into dishes (**transform**), and are served at the table (**load**). When orders pile up, the kitchen does not make guests stand next to the stove: they get a table number, and the food is brought over later. That is the big picture of this meeting.

Part 1

Reports and Data Aggregation

Four Basic Terms

ETL (Extract, Transform, Load): a common pattern: take raw data, reshape it, then store or present it in a new form.

Batch Processing: processing lots of data together in groups, not one record at a time.

Queue: a mechanism for handing heavy work to a separate process that runs it in the background, instead of during the user's request.

Import/Export: bringing data in from an external format (CSV, Excel) into the database, or sending it out to an external format to be read outside the app.

List vs Report: A Pile of Receipts vs a Month-End Summary

A list page is like a pile of receipts in the cash drawer: everything is there, but you have to add it up yourself. A report page is like the month-end summary: already totaled and ready to read. The question is narrower but heavier: "what is the total of X over a given period".

```
$totalKata = Article::whereBetween('published_at', [
    $from, $to,
])->sum('word_count');
```

`whereBetween` keeps rows whose date falls between two bounds, then `sum()` adds up one column of the filtered result. The database does the adding, so Laravel does not need to load thousands of rows into memory just to total them.

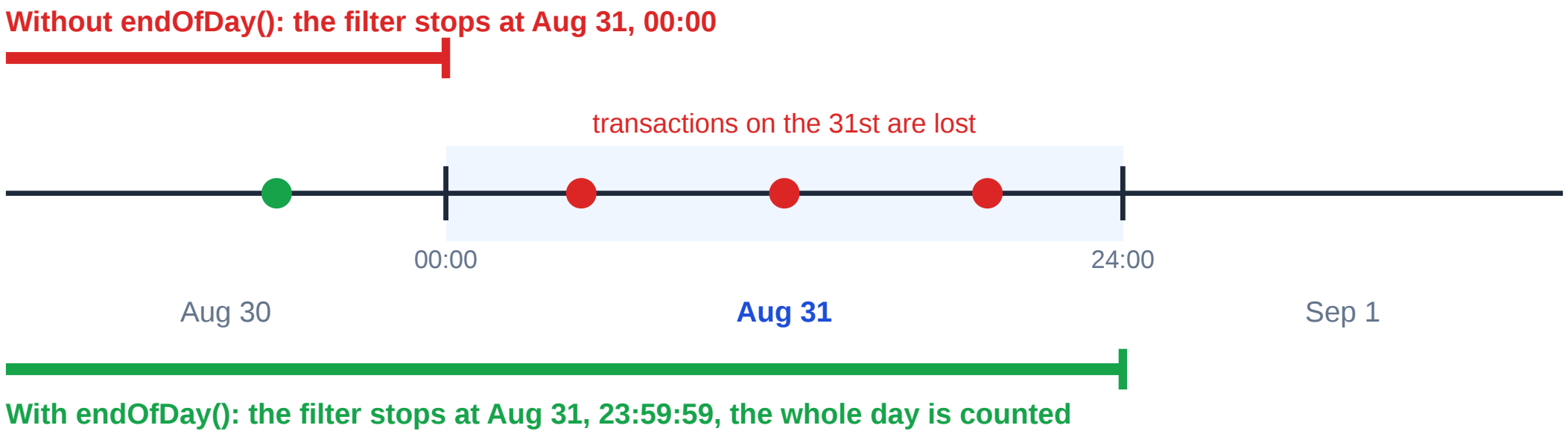
Common Pitfall: The Day That Gets Cut Off

> A filter of "August 1 to August 31" taken literally means "up to 00:00:00 on the 31st". Every transaction made during the 31st disappears, like a shop that tallies August revenue but stops counting the moment it opens its doors on the last day.

```
$from = $request->date('from')->startOfDay();
$to = $request->date('to')->endOfDay();
```

`startOfDay()` turns the start date into 00:00:00, and `endOfDay()` turns the end date into 23:59:59, so the whole last day is counted.

The Day-Boundary Pitfall on a Timeline



Stored Totals Are Not Recomputed

Meeting 6 made the point: a value like a transaction total must not be trusted when it comes from client input, and must be recomputed on the server when the transaction is created. Once it is stored correctly, a report can use it as is. It is like the total printed on a receipt: when summarizing, you add up the receipt totals instead of re-checking every item on them.

```
// The report just sums the stored column
$totalPenjualan = Transaction::whereBetween('created_at', [$from, $to])
    ->sum('total');
```

> Recomputing the total from detail rows every time the report opens burdens the server for no reason: the number has been correct since the transaction was created.

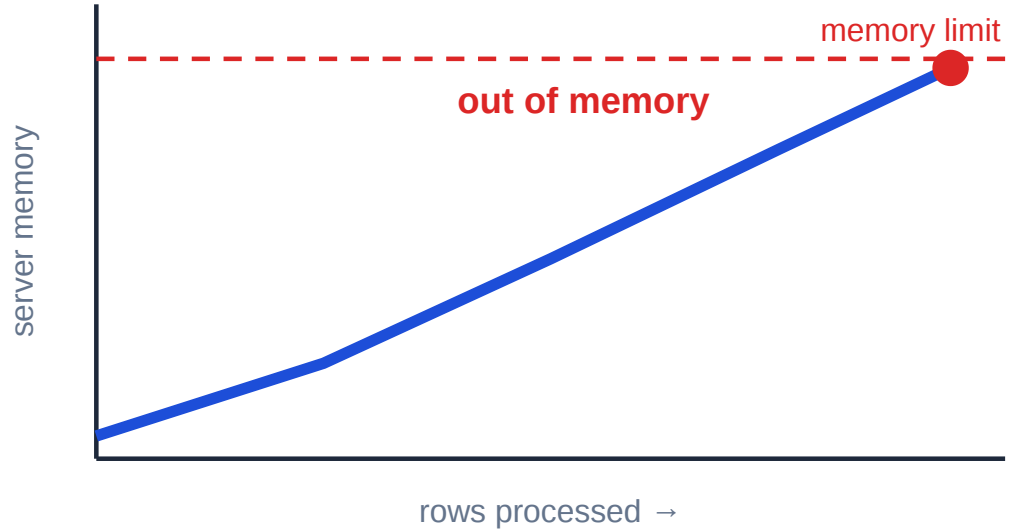
Part 2

CSV Export and Import

Bucket vs Tap: Two Ways to Send a Big File

Bucket: collect first, send later

get() then build one big string



Tap: let it flow row by row

lazyById() + streamDownload()



Export with `streamDownload()` and `lazyById()`

The tap way: fetch data in small chunks, write it to the output, then forget it.

```php
$articles = Article::with('author')
    ->whereBetween('published_at', [$from, $to]);

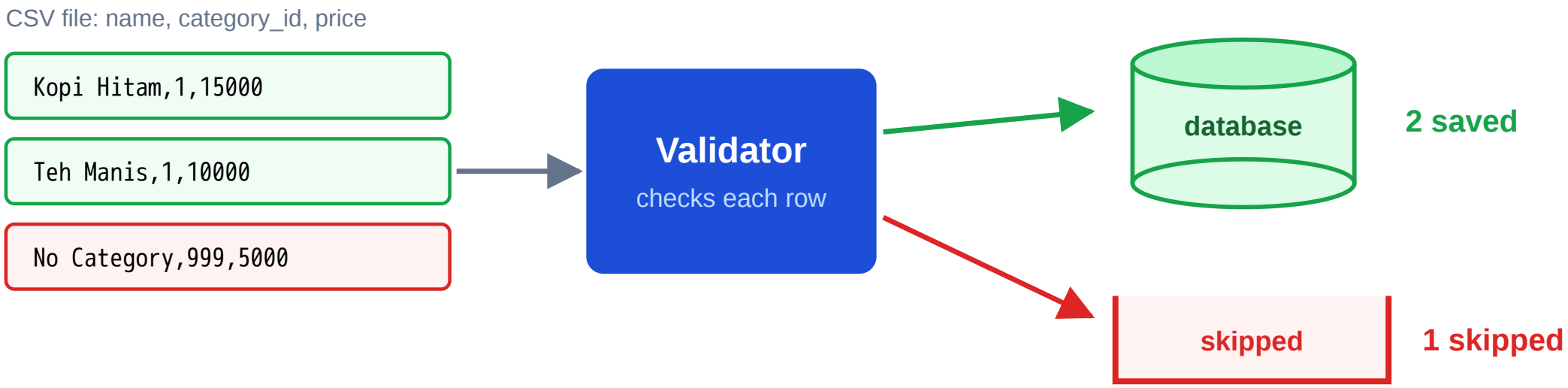
return response()->streamDownload(function () use ($articles) {
    $out = fopen('php://output', 'w');
    fputcsv($out, ['Title', 'Author', 'Date']);
    foreach ($articles->lazyById() as $a) {
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);
    }
    fclose($out);
}, 'articles-report.csv');
```

`lazyById()` reads the database in chunks, and `streamDownload()` sends each row as it goes, so memory stays flat.

> Laravel docs: `HTTP Responses → File Downloads` and `Eloquent → Chunking / lazyById()`

Import: Like a Cashier Scanning Groceries

Import goes the other way: read a CSV file, then turn each of its rows into a new database row. Think of a cashier scanning groceries. If one barcode is damaged, the cashier sets that item aside and keeps scanning the rest, instead of sending every customer home.



one bad row does not stop the others

Import: Validate Each Row, Skip the Bad Ones

```php
$rows = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$rows = array_map('str_getcsv', $rows);
$header = array_shift($rows);

foreach ($rows as $row) {
    $validator = Validator::make(array_combine($header, $row), [
        'title' => ['required', 'string'],
        'author_id' => ['required', 'exists:authors,id'],
    ]);

    if ($validator->fails()) {
        continue; // skip this row, move on to the next one
    }

    Article::create($validator->validated());
}
```

CSV saved from Excel with an Indonesian locale often uses a semicolon (`;`) as the separator and adds an invisible character (BOM) at the start of the file. The first header then goes unrecognized and every row fails validation. Check the header first if every row is suddenly rejected.

Part 3

When to Use a Queue

Two Ways to Process Work

Directly (inside the request)

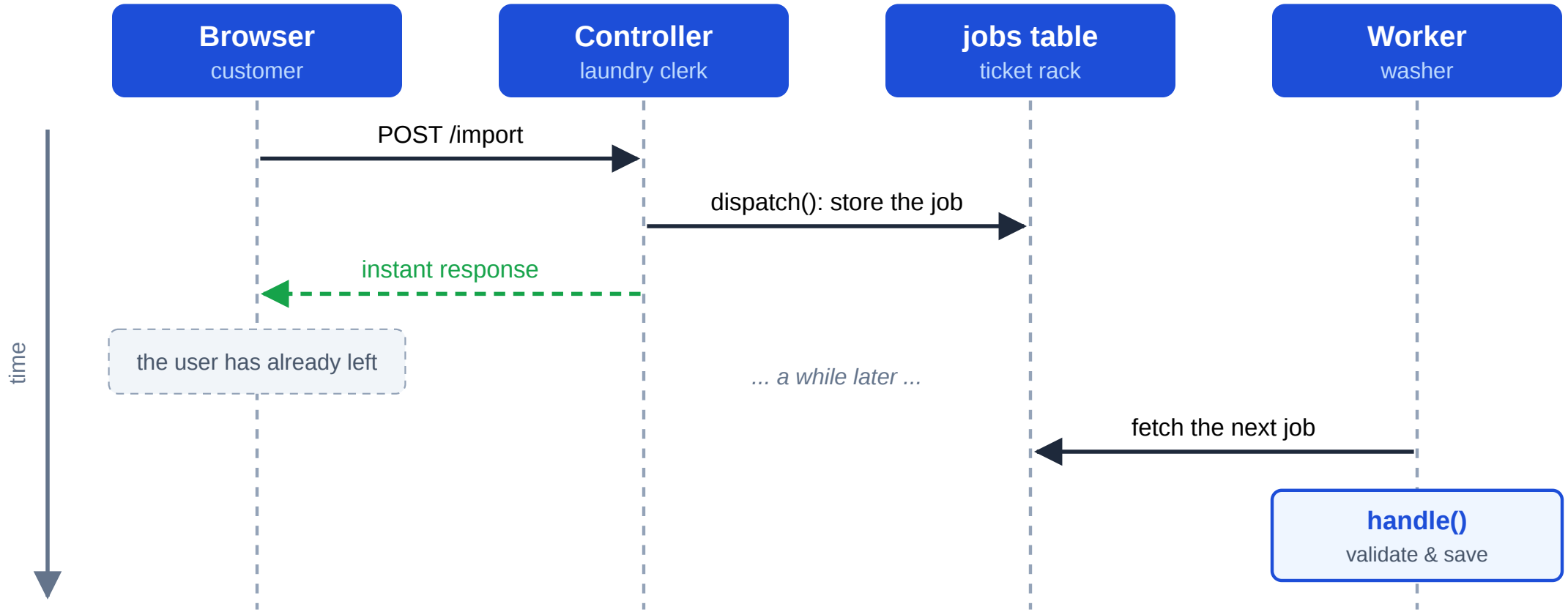
- The user waits until it finishes
- If it fails, the user knows right away
- Fits short tasks whose result is needed immediately

Via a queue (job)

- Instant response, work runs in the background
- The user does not wait in front of the browser
- Fits heavy tasks or tasks with unpredictable duration

> Laundry analogy: one T-shirt, you gladly wait at the counter. A whole sack, you drop it off, get a ticket, and go home. Importing a few dozen rows is one T-shirt. Importing thousands of rows is a whole sack, and HTTP requests have a timeout.

A Queue = A Laundry Drop-Off



The Job Flow in Laravel




```
php artisan make:job ProcessArticleImport
```

```
ProcessArticleImport::dispatch($csvPath);

return back()->with('success', 'The import is being processed in the background.');
```

The controller only saves the file and sends one job to the queue, then responds to the user right away. The real work (the validation and saving loop) moves into the `handle()` method of that job class.

Worker: The Person Who Works Through the Ticket Pile

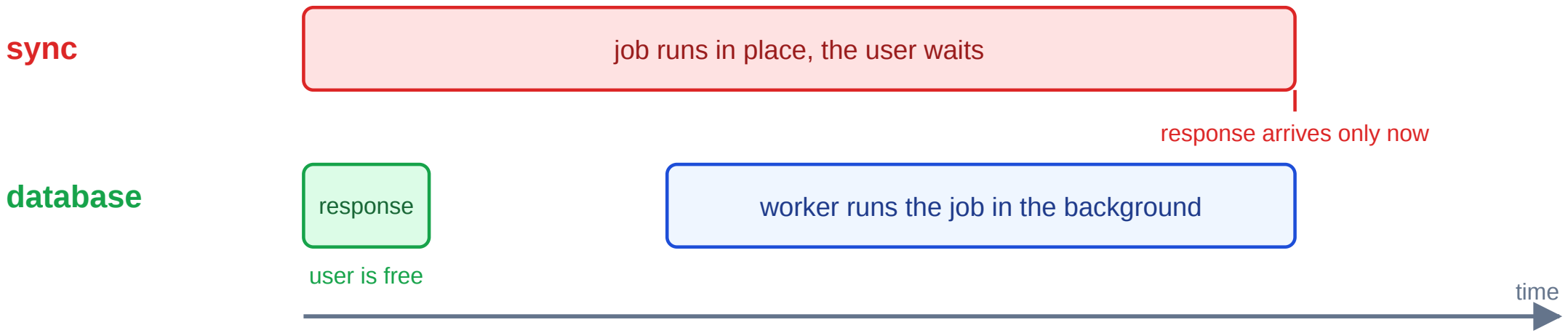
> **Worker:** a separate PHP process that keeps running, takes jobs from the `jobs` table one at a time, and runs each one's `handle()` method.

```
php artisan queue:work
```

Without a worker, jobs just pile up in the `jobs` table. It is a laundry with no one to wash: tickets keep growing, but no laundry gets done, because `dispatch()` only stores the ticket, it does not wash anything.

> If `QUEUE_CONNECTION` in `.env` says `sync`, the job runs right there and never enters a real queue. It is like a clerk telling you to wait until the laundry is dry even though you are already holding a ticket.

`sync` vs `database`: Who Waits?



Signs a Task Belongs in a Queue

- Its duration cannot be predicted in advance, since it depends on the size of the data
- The user does not need to see the result immediately on the same page
- It calls a slow external service, such as sending email or calling a third-party API

> A report that is opened and read right on the page is usually still better done directly, because the user is waiting for the result at that moment. A queue is not the answer to every heavy task, only to those that do not need an instant result.

When a Job Fails: The Laundry Complaint Book

Problem laundry is not quietly thrown away. A failed job is recorded in the `failed_jobs` table along with its error message, and can be retried once the problem is fixed.

```
php artisan queue:failed        # list failed jobs
php artisan queue:retry all     # retry all of them
```

```
class ProcessArticleImport implements ShouldQueue
{
    use Queueable;

    public int $tries = 3; // at most 3 attempts
}
```

> If a job "ran but produced nothing", check `queue:failed` first: the error message is almost always there.

> Laravel docs: `Queues → Dealing With Failed Jobs`

Applying It to Simple POS

You will apply these report, CSV export/import, and queue concepts directly to Simple POS in the lab jobsheet: build a sales report with a date filter, export it to CSV, build a product import from CSV, then move that import into a queued job so the user does not wait in front of the browser.

> Full code: `github.com/se-polinema/simple-pos-ch08`

Summary (1/2)

- Reports summarize data over a date range with `whereBetween`, and the aggregation (`sum()`) is best done by the database
- `startOfDay()` / `endOfDay()` keep the last day from being cut off at midnight
- Values stored correctly (like a transaction total) can be used as is, with no need to recompute them every time a report opens
- CSV export uses `lazyById()` and `streamDownload()`: a tap, not a bucket, so memory stays flat

Summary (2/2)

- CSV import validates each row and skips failures with `continue`, instead of stopping the whole process
- Work with unpredictable duration or large volume belongs in a queued job, like a laundry drop-off: instant response, work follows
- A dispatched job only enters the `jobs` table, and runs only once a worker (`queue:work`) processes it
- Failed jobs are recorded in `failed_jobs` and can be retried with `queue:retry`

References & Discussion

Official Laravel documentation (Queues, File Storage, HTTP Responses)

Full code: `github.com/se-polinema/simple-pos-ch08`

Next meeting: Designing and Building a REST API